Università degli Studi dell'Aquila

Black Box Testing con LLM

Black Box Testing con LLM: Tecniche e Approcci

Autore: Antonio Sabatini
Coautore: Marco Nicolella
Supervisori: Henry Muccini, Roberta Capuano
Corso: Informatica Triennale
Anno: 2025

SOMMARIO

Questo report esplora l'integrazione e l'impatto dei **Large Language Models (LLM)** nel contesto del **black box testing**. L'obiettivo primario è valutare come gli LLM possano ottimizzare e automatizzare la generazione di test case, arricchendo al contempo le metodologie di testing software esistenti.

Il documento introduce le basi del **prompt engineering**, illustrando le tecniche più efficaci per massimizzare le capacità degli LLM (come Zero-shot, Few-shot, Chain-of-Thought e Prompt Refinement). Vengono inoltre delineate le **best practice** essenziali per l'applicazione degli LLM nel testing, enfatizzando la necessità di una profonda comprensione del dominio applicativo, l'iterazione dei prompt e la cruciale revisione umana dei risultati.

Successivamente, vengono analizzati tre approcci metodologici distinti: **LLM-Driven**, **Human-Driven** e **Hybrid**, con un'esplorazione dettagliata dei loro sottocasi e dei relativi flussi di lavoro. Attraverso una matrice di valutazione comparativa, che considera efficienza, qualità, affidabilità e costi di formazione, si identifica l'approccio **LLM Majority (70-30)** come la soluzione più bilanciata e promettente per la produzione di documentazione di testing di alta qualità.

Infine, il report fornisce un **esempio di prompt strutturato** per il sistema "Podcast di Ateneo", dimostrando l'applicazione pratica delle tecniche discusse e proponendo una serie di riferimenti utili per approfondire l'argomento. Gli LLM si confermano uno strumento potente per accelerare il testing, ma la supervisione e l'expertise umana rimangono indispensabili per garantire l'accuratezza e la rilevanza dei test generati.

Indice

1 CONOSCENZE BASE SUL PROMPT ENGINEERING **3**
1.1 Zero-shot (Prompt Diretto) . . . 3
1.2 Few-shot (Prompt con Esempi) . . . 3
1.3 Chain-of-Thought (CoT) e Tree-of-Thought (ToT) . . . 3
1.4 Prompt Refinement . . . 4

2 BEST PRACTICE FOR LLM IN SOFTWARE TESTING **5**
2.1 Comprendere l'Applicazione . . . 5
2.2 Prompt Engineering Efficace . . . 5
2.3 Iterazione e Raffinamento del Prompt . . . 5
2.4 Utilizzo di Variabili e Condizioni . . . 5
2.5 Post-Elaborazione e Revisione Umana . . . 5
2.6 Variare gli Stili di Test . . . 6
2.7 Gestione delle Limitazioni . . . 6
2.8 Structured Prompts (JSON/XML) . . . 6

3 POSSIBILI APPROCCI **7**
3.1 Obiettivo principale . . . 7
3.2 Albero Dei Path con Interconnessioni . . . 7
3.3 Esempi di Percorsi Ibridi Possibili . . . 8
3.4 PATH 1: LLM-DRIVEN APPROACH . . . 9
3.4.1 Full Automation . . . 9
3.4.2 LLM Generation . . . 9
3.5 PATH 2: HUMAN-DRIVEN APPROACH . . . 9
3.5.1 Pure Manual . . . 9
3.5.2 Expert Review . . . 9
3.5.3 Traditional Methods . . . 10
3.6 PATH 3: HYBRID APPROACH . . . 10
3.6.1 LLM Majority (70-30) . . . 10
3.6.2 Human Majority (30-70) . . . 10
3.6.3 Balanced Integration (50-50) . . . 10
3.7 Valutazione dei Percorsi . . . 11
3.8 Criteri di Valutazione . . . 11
3.9 Matrice di Valutazione . . . 11
3.10 Conclusioni Analitiche . . . 11
3.11 Percorso Raccomandato . . . 11
3.12 Considerazioni Finali . . . 11

4 ESEMPIO DI PROMPT BEN STRUTTURATO [1]
(Il prompt comprende tutta la sezione) **12**
4.1 ISTRUZIONI: . . . 12
4.2 TECNICHE DA APPLICARE . . . 12
4.3 OUTPUT ATTESO . . . 12
4.4 FUNZIONALITÀ DEL SISTEMA DA CONSIDERARE . . . 13

5 REFERENCES **14**

1 CONOSCENZE BASE SUL PROMPT ENGINEERING

Il **Prompt Engineering** è una disciplina emergente e cruciale nell'interazione con i **Large Language Models (LLM)**. Consiste nella pratica di ideare, formulare e ottimizzare gli input (*prompt*) forniti al modello per guidarne il comportamento, ottenere risposte accurate, pertinenti e nel formato desiderato. L'obiettivo principale è sfruttare appieno le capacità dell'LLM, trasformandolo da generatore generico di testo in uno strumento specializzato per compiti specifici.

In generale, il prompt engineering si concentra su:

- **Strutturazione Chiara e Precisa:** Definire il contesto, il ruolo dell'LLM (ad esempio, "Sei un esperto di storia") e i requisiti specifici del compito.
- **Fornitura di Contesto e Vincoli:** Includere informazioni pertinenti, esempi di input/output o regole precise che il modello deve considerare.
- **Iterazione e Ottimizzazione:** Raffinare progressivamente i prompt analizzando i risultati ottenuti, migliorando la qualità e la coerenza dell'output.

Quando applicato al **black box testing**, il prompt engineering assume un ruolo determinante. Permette ai tester di sfruttare le capacità dell'LLM di comprendere il linguaggio naturale e generare contenuti coerenti, variati e pertinenti, accelerando significativamente le fasi di progettazione e documentazione dei test.

1.1 Zero-shot (Prompt Diretto)

- **Descrizione:** L'approccio Zero-shot consiste nel fornire all'LLM una descrizione del compito senza alcun esempio specifico di input/output. Il modello si affida unicamente alla sua conoscenza pre-addestrata per generare la risposta.
- **Esempio Generale:** "Scrivi una breve poesia sulla luna." L'LLM utilizzerà la sua conoscenza generale e creatività per produrre un testo originale.
- **Riferimento:** [4]

1.2 Few-shot (Prompt con Esempi)

- **Descrizione:** Si forniscono all'LLM da 1 a 5 esempi di input/output completi all'interno del prompt, permettendogli di apprendere il formato, lo stile e la logica desiderata.
- **Esempio Generale:** Fornire due test cases già formattati per il login e chiedere all'LLM di produrne altri seguendo lo stesso schema.
- **Riferimento:** [5]

1.3 Chain-of-Thought (CoT) e Tree-of-Thought (ToT)

- **CoT:** L'LLM esplicita il ragionamento passo-passo prima di fornire la risposta, migliorando la qualità dei test generati.
- **ToT:** Estensione del CoT che esplora percorsi multipli in parallelo, utile per considerare scenari complessi o multi-variabile.
- **Applicazione nel Testing:** Permette di generare test cases che considerano interazioni tra più componenti o condizioni limite complesse.
- **Riferimento:** [5]

1.4 Prompt Refinement

- **Descrizione:** Processo iterativo di analisi e ottimizzazione dei prompt in base ai risultati ottenuti dall'LLM.
- **Applicazione nel Testing:** Migliora progressivamente la qualità, la pertinenza e la completezza dei test generati. Questo è fondamentale per ridurre errori, duplicazioni e test poco realistici.
- **Riferimento:** [5]

2 BEST PRACTICE FOR LLM IN SOFTWARE TESTING

L'uso degli LLM (Large Language Models) per generare test di black box sta diventando sempre più diffuso, offrendo un modo rapido per creare test cases diversificati. Ecco alcune pratiche consigliate per sfruttarli al meglio:

2.1 Comprendere l'Applicazione

Anche se si tratta di black box testing, avere una comprensione base della funzionalità del software è cruciale. Solitamente gli LLM più contesto riceveranno, migliori saranno i test generati. Dedica un minimo di tempo a capire lo scopo principale e le interazioni chiave dell'applicazione.

2.2 Prompt Engineering Efficace

Il prompt è il cuore della generazione. Sii specifico e dettagliato. Includi:

- **Ruolo:** Indicalo come "un tester software esperto in black box testing".
- **Obiettivo:** "Genera test cases per la funzionalità X del software Y."
- **Formato:** Specifica il formato desiderato (es. tabella con ID, Descrizione, Passi, Risultato Atteso).
- **Contesto:** Fornisci dettagli rilevanti sull'input, output e vincoli.
- **Esempi (Few-shot prompting):** Se possibile, includi 1-2 esempi di test cases ben formattati. Questo guida l'LLM sul tipo di output desiderato.

2.3 Iterazione e Raffinamento del Prompt

Difficilmente otterrai il prompt perfetto al primo tentativo. Itera e raffina. Se i risultati non sono soddisfacenti, analizza cosa manca o è ambiguo nel prompt e modificalo. Prova a suddividere prompt complessi in prompt più piccoli per ottenere risultati intermedi migliori. In questa fase è fondamentale una buona conoscenza delle tecniche di testing che ti permettano di identificare eventuali imperfezioni e guidare l'LLM verso un output migliore. L'uso degli LLM non riesce a sostituire completamente l'uomo.

2.4 Utilizzo di Variabili e Condizioni

Per test più dinamici, puoi usare variabili e condizioni nel prompt. Ad esempio, "Genera test per l'input di un campo età, considerando valori interi tra 0 e 110." Questo permette all'LLM di pensare in termini di intervalli e validazioni.

2.5 Post-Elaborazione e Revisione Umana

L'output dell'LLM è un ottimo punto di partenza, ma non una soluzione finale. Revisiona sempre i test generati.

- **Rimuovi duplicati:** Gli LLM possono generare test simili.
- **Correggi imprecisioni:** Verifica la coerenza logica e la fattibilità.
- **Aggiungi specificità:** Integra dettagli che solo un essere umano può cogliere (es. dati di test specifici non evidenti dal prompt).
- **Prioritizza:** Non tutti i test generati avranno la stessa importanza.

2.6 Variare gli Stili di Test

Non limitarti a un solo tipo di test. Chiedi all'LLM di generare:

- **Test di happy path:** Scenari d'uso tipici.
  - Inserimento corretto di dati in un form (nome, email, password) e registrazione riuscita.
  - Aggiunta di un prodotto al carrello e completamento dell'ordine senza errori.
- **Test di edge case:** Valori limite, input anomali.
  - Inserimento di stringhe vuote o estremamente lunghe nei campi di testo.
  - Numeri molto grandi o negativi in campi numerici.
  - Date limite (es. 29 febbraio in un anno non bisestile).
- **Test di errore/invalid input:** Dati non validi, formati errati.
  - Email senza "@" o con caratteri speciali non permessi.
  - Password troppo corta o senza caratteri richiesti.
  - Formati di data o numeri errati.
- **Test di performance (base):** Scenari che potrebbero stressare il sistema.
  - Creare simultaneamente 100 utenti e verificare tempi di risposta.
  - Caricare un file molto grande e misurare il tempo di upload.
- **Test di sicurezza (base):** Input malevoli noti.
  - SQL injection semplice in un campo di login (‘’ OR ’1’=’1‘).
  - Script di tipo XSS in un campo di commento (‘¡script¿alert(’test’)¡/script¿‘).
  - Test di accesso non autorizzato a pagine riservate.

2.7 Gestione delle Limitazioni

Sii consapevole che gli LLM possono:

- **Allucinare:** Generare informazioni o test non corretti.
- **Essere non deterministici:** Rispondere in modo diverso allo stesso prompt.
- **Mancare di ragionamento profondo:** Difficoltà con logiche complesse o scenari molto specifici senza un contesto adeguato.

2.8 Structured Prompts (JSON/XML)

- **Descrizione:** Si richiede all'LLM di produrre l'output in un formato strutturato (es. JSON, XML, YAML) per renderlo leggibile da macchine e integrabile con framework di test.
- **Applicazione:** Essenziale per test automatici, ad esempio per API REST, dove l'LLM può generare oggetti JSON con dettagli su metodo, endpoint, body, status code attesi.

Queste tecniche, spesso usate in combinazione, massimizzano l'efficacia degli LLM nel black-box testing.

3 POSSIBILI APPROCCI

Analiziamo i possibili percorsi metodologici per il raggiungimento di una documentazione ottimale nel contesto del black box testing, con particolare focus sull'integrazione dei Large Language Models (LLM) nei processi di testing.

3.1 Obiettivo principale

L'obiettivo primario è la realizzazione di una **soluzione ottima** per la documentazione del black box testing, intesa come la produzione di documentazione di alta qualità che copra efficacemente tutte le forme di testing black box.

3.2 Albero Dei Path con Interconnessioni

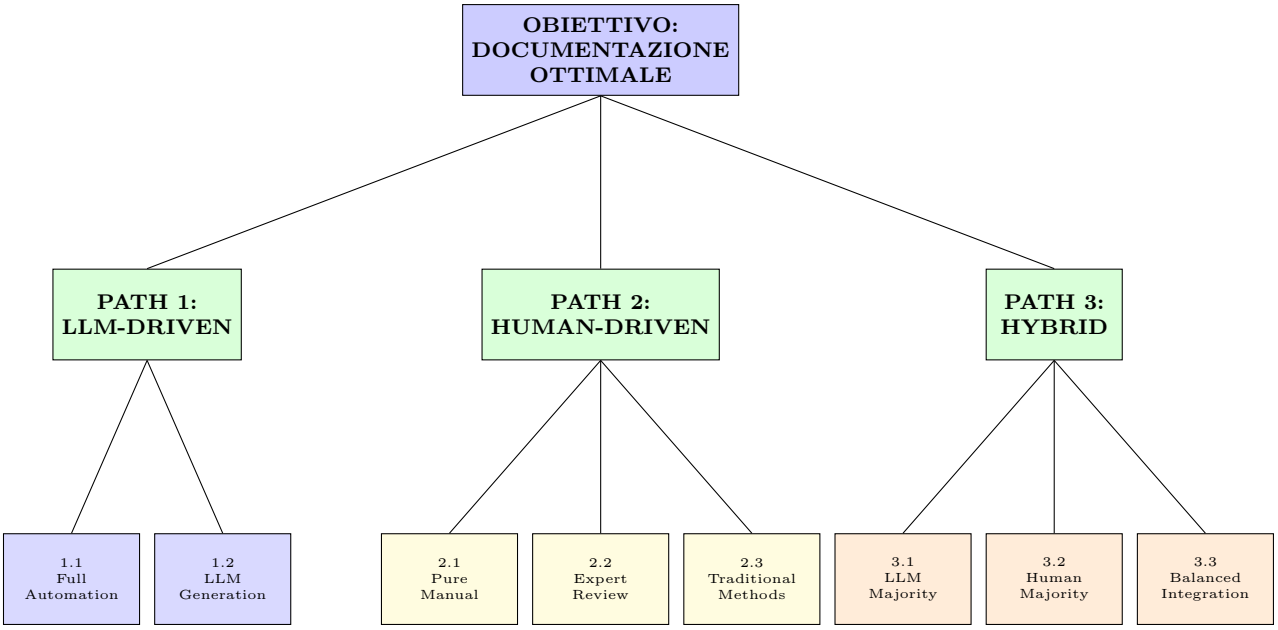


Figura 1: Albero delle possibili vie per il Black Box Testing con LLM - Versione Interconnessa

3.3 Esempi di Percorsi Ibridi Possibili

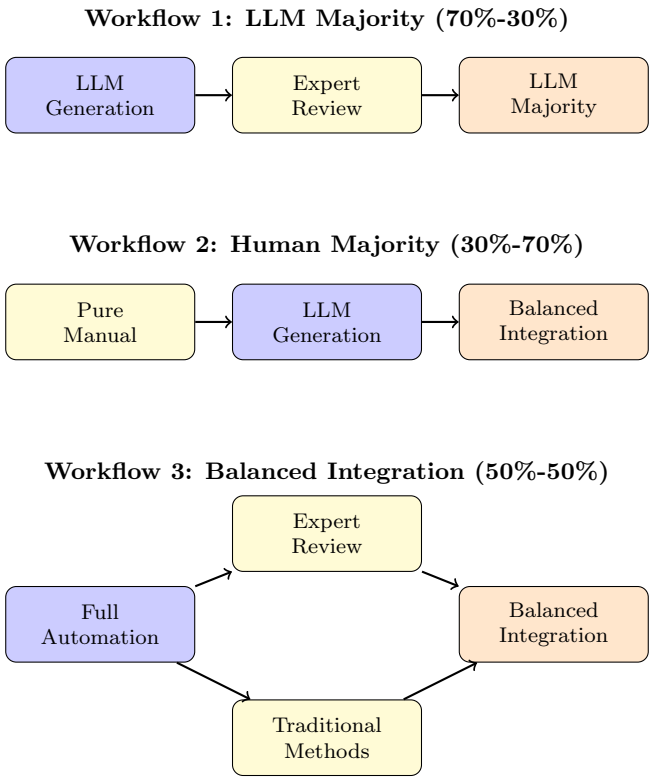


Figura 2: Esempi di Workflow Ibridi con Percentuali di Contributo

3.4 PATH 1: LLM-DRIVEN APPROACH

3.4.1 Full Automation

Approccio: L'LLM gestisce tutto il processo senza intervento umano significativo.

- Input: Specifiche software
- LLM genera automaticamente: test cases, test data, scenari, documentazione
- Output: Documentazione completa senza revisione umana

Rischi: Possibilità di generare test non realistici o perdere contesto

3.4.2 LLM Generation

Approccio: LLM genera contenuti, l'uomo supervisiona e valida il lavoro.

- LLM genera bulk di test cases
- L'uomo rivede e approva/scarta
- Focus su quantità e copertura
- Minimal human effort per la validazione

Esempio pratico: LLM genera 100 test cases per modulo login → L'uomo rivede in 30 minuti, scarta 20 non pertinenti → Risultato: 80 test cases validati rapidamente.

3.5 PATH 2: HUMAN-DRIVEN APPROACH

3.5.1 Pure Manual

Approccio: Tutto progettato e documentato manualmente, zero LLM.

- Analisi requisiti manuale
- Design test cases basato su esperienza
- Scrittura documentazione tradizionale
- Metodologie consolidate per lo sviluppo di documentazione BBT

Esempio pratico: Senior tester analizza specifiche per 2 giorni → Progetta 25 test cases critici basati su esperienza → Documenta manualmente in 1 settimana → Risultato: qualità alta ma processo lento.

3.5.2 Expert Review

Approccio: Team di esperti rivede e valida tutto collaborativamente.

- Design iniziale da expert lead
- Peer review sistematico
- Multiple validation rounds
- Quality gates rigorose

Esempio pratico: Expert 1 progetta test suite → Expert 2 e 3 reviewano indipendentemente → Meeting di validazione collettiva → Iterazioni fino a consensus.

3.5.3 Traditional Methods

Approccio: Applicazione rigorosa di tecniche BBT classiche. **Tecniche specifiche**:

- **Equivalence Partitioning**: dividere input in classi equivalenti
- **Boundary Value Analysis**: testare valori limite
- **Decision Table Testing**: tabelle decisionali per logica complessa

Esempio pratico: Campo "età" (1-120) → Equivalence: valido (1-120), invalido (¡1, ¿120) → Boundary: 0,1,2 / 119,120,121 → Decision table per combinazioni età+status.

3.6 PATH 3: HYBRID APPROACH

3.6.1 LLM Majority (70-30)

Approccio: LLM fa il lavoro pesante, l'umano controlla aspetti critici.

1. LLM genera gran parte della documentazione
2. L'uomo identifica i gap critici (30 effort)
3. LLM integra feedback umano
4. L'uomo approva la versione finale

Esempio pratico: LLM genera 80 test cases in 10 minuti → L'uomo identifica 5 scenari business-critical mancanti in 30 minuti → LLM aggiunge gli scenari specificati dall'umano → L'uomo effettua una final review in 20 minuti → Totale: 1 ora vs 8 ore manual.

3.6.2 Human Majority (30-70)

Approccio: L'uomo progetta una strategia, LLM la supporta e migliora.

1. L'uomo definisce la documentazione
2. LLM ottimizza le formulazioni e aggiunge varianti
3. L'uomo mantiene controllo decisionale

Esempio pratico: Senior tester progetta 20 test cases core in 4 ore → LLM suggerisce 15 varianti e migliora descrizioni in 30 minuti → L'umano seleziona 8 varianti utili e finalizza in 1 ora → Risultato: qualità human + efficiency boost.

3.6.3 Balanced Integration (50-50)

Approccio: Collaborazione continua e paritaria.

- Iterazioni continue human-LLM
- Cross-validation reciproca
- Decisioni congiunte
- Feedback loop costante

Esempio pratico: Ciclo iterativo → 1) L'uomo progetta lo scenario A, LLM lo valida e suggerisce miglioramenti → 2) LLM propone lo scenario B, l'uomo lo valida e rifinisce → 3) Validazione incrociata continua → 4) Convergenza verso una soluzione ottimale.

3.7 Valutazione dei Percorsi

3.8 Criteri di Valutazione

I criteri utilizzati per la valutazione comparativa sono:

- **Efficienza**: Tempo e risorse richieste
- **Qualità**: Completezza e accuratezza della documentazione
- **Affidabilità**: Consistenza e riproducibilità dei risultati
- **Formazione LLM**: Costo dell'investimento formativo sull'uso degli LLM
- **Formazione BBT**: Costo dell'investimento formativo sulle tecniche BBT

3.9 Matrice di Valutazione

I punteggi hanno un valore che va da 1(scarso) a 5 (ottimo.)

Il punteggio totale viene calcolato come segue: [eff+qual+aff]-[C(LLM)+C(BBT)], con un valore massimo di 13 dato da 15-2

Percorso	Efficienza	Qualità	Affidabilità	C(LLM)	C(BBT)	Totale
1.1 Full Automation	5	2	3	5	1	**4/13**
1.2 LLM Generation	4	3	3	4	2	**4/13**
2.1 Pure Manual	1	4	4	1	5	**3/13**
2.2 Expert Review	2	5	5	1	5	**6/13**
2.3 Traditional Methods	2	4	3	1	5	**3/13**
3.1 LLM Majority	4	4	4	3	2	**7/13**
3.2 Human Majority	2	4	4	2	4	**4/13**
3.3 Balanced Integration	3	4	4	3	3	**5/13**

Tabella 1: Matrice di Valutazione Comparativa

3.10 Conclusioni Analitiche

3.11 Percorso Raccomandato

L'analisi quantitativa evidenzia il **PATH 3.1 - LLM Majority (70%-30%)** come soluzione ottimale, con un punteggio totale di 7/13. Questo approccio:

- Ottimizza l'efficienza mantenendo controllo qualitativo
- Garantisce un bilanciamento ottimale del rapporto costo-beneficio
- Grazie alla supervisione umana, è possibile mantenere un approccio meno rigido con gli LLM
- Non è necessaria una conoscenza approfondita delle tecniche di BBT poichè l'umano viene guidato dagli LLM

3.12 Considerazioni Finali

- L'approccio puramente automatizzato (1.1) presenta rischi di qualità elevati
- L'approccio puramente manuale (2.1, 2.2) non sfrutta le potenzialità emergenti degli LLM
- Gli approcci ibridi dimostrano superiore robustezza complessiva
- L'approccio basato sulla collaborazione di esperti mantiene un punteggio molto alto poichè rappresenta la massima espressione di qualità ed Affidabilità ma ha una efficenza davvero bassa,

4 ESEMPIO DI PROMPT BEN STRUTTURATO [1] (Il prompt comprende tutta la sezione)

CONTESTO

Sei un esperto di test software incaricato di eseguire black box testing su un progetto "Podcast di Ateneo" che mira a creare una piattaforma di podcast dedicata all'Ateneo con l'obiettivo di fornire contenuti accademici, culturali e informativi agli studenti, ai docenti e al pubblico in generale. Il podcast offrirà una nuova modalità di apprendimento e condivisione di conoscenze. Le funzionalità centrali includono la possibilità per gli utenti autorizzati, che comprendono docenti, studenti e ospiti, di registrare e caricare i loro podcast sulla piattaforma. I podcast saranno ordinati per categorie, quali corsi, eventi, interviste e altro, per facilitare la navigazione e la ricerca. Gli utenti potranno effettuare ricerche avanzate per trovare specifici podcast, argomenti o relatori. Sarà anche possibile abbonarsi a podcast specifici o a categorie di interesse e ricevere notifiche sugli aggiornamenti. L'interazione e la partecipazione della comunità saranno promosse attraverso la possibilità di commentare e fornire feedback sui podcast. Inoltre, i podcast potranno essere condivisi su piattaforme di social media per aumentarne la visibilità e l'interazione.

4.1 ISTRUZIONI:

Approccio Tree-of-Thought

Utilizzando l'approccio Tree-of-Thought (ToT), sviluppa test cases completi per il black-box testing del sistema. Per ogni tecnica di black-box testing specificata, dovrai:

1. Comprendere la tecnica: Definisci brevemente in cosa consiste la tecnica
2. Analizzare l'applicabilità: Considera come si applica al nostro sistema
3. Esplorare molteplici percorsi di test: Pensa a diversi rami di analisi possibili
4. Valutare ogni percorso: Determina quali percorsi sono più promettenti e perché
5. Selezionare e sviluppare i test cases: Dettagliare i test cases risultanti.

4.2 TECNICHE DA APPLICARE

Applica la seguente tecnica di black box testing al sistema "Podcast di Ateneo":

1. **Category Partitioning**
   - Identifica i parametri e le funzionalità chiave del sistema
   - Per ogni parametro, individua categorie di input significativamente diverse
   - Definisci vincoli tra le categorie
   - Crea combinazioni di test rappresentative tenendo conto delle best practice necessarie

4.3 OUTPUT ATTESO

Fornisci:

1. Una tabella dettagliata delle combinazioni di test risultanti, strutturati secondo questo formato:
   a) ID Test
   b) Descrizione
   c) Input/Azioni
   d) Risultato atteso

4.4 FUNZIONALITÀ DEL SISTEMA DA CONSIDERARE

1. **Utenti Autorizzati (Registrazione e Caricamento):**
   - Tipi di Utente: considera i diversi ruoli degli utenti autorizzati (Docente, Studente, Ospite) e anche utenti non autorizzati per testare i controlli di accesso.
   - Azioni di Caricamento: analizza sia i caricamenti riusciti che i tentativi falliti dovuti a diverse cause (formato file non valido, dimensione eccessiva, mancanza di autorizzazione, metadati incompleti).
   - Stato del Podcast: considera i diversi stati in cui un podcast può trovarsi dopo il caricamento (Pubblicato, In bozza, In moderazione).
   - Scenari Limite: includi il test di caricamenti multipli simultanei per verificare la gestione del carico.
2. **Categorie dei Podcast:**
   - Navigazione: considera la navigazione attraverso le categorie principali e le sottocategorie, inclusi i casi di categorie vuote.
   - Stato della Categoria: includi test per verificare la visibilità dei podcast in base allo stato attivo o disattivo della categoria.
   - Categorie Inesistenti: considera come il sistema gestisce la navigazione o la ricerca di categorie non presenti.
3. **Navigazione e Ricerca:**
   - Ricerca Semplice: considera la ricerca per titolo, relatore e parole chiave, includendo corrispondenze esatte e parziali, e anche ricerche senza input.
   - Ricerca Avanzata: analizza la ricerca combinando più criteri (categoria + parola chiave, relatore + data, ecc.) e l'utilizzo di filtri specifici.
   - Risultati di Ricerca: considera sia i casi in cui la ricerca produce risultati multipli sia quelli in cui non trova corrispondenze.
4. **Abbonamenti e Notifiche:**
   - Azioni di Abbonamento: considera l'abbonamento a podcast specifici e a intere categorie, inclusi tentativi di abbonamento a elementi inesistenti e abbonamenti multipli.
   - Rimozione Abbonamenti: includi il test della funzionalità di disiscrizione.
   - Notifiche: considera la ricezione di notifiche per nuovi episodi e nuovi podcast nelle categorie abbonate, e anche la configurazione delle preferenze di notifica.
   - Scenari Negativi: includi casi in cui le notifiche potrebbero non essere ricevute (es. errore di sistema).
5. **Interazione e Partecipazione (Commenti e Feedback):**
   - Pubblicazione Commenti: considera la pubblicazione di commenti validi, vuoti, con formattazione e con link/immagini (tenendo conto della moderazione).
   - Autenticazione: includi il test del requisito di essere loggati per commentare.
   - Moderazione: analizza i processi di approvazione e rifiuto dei commenti.
   - Feedback: considera l'invio di feedback generale e specifico sui podcast.
   - Valutazione: includi la valutazione dei podcast.
6. **Condivisione sui Social Media:**
   - Piattaforme Diverse: considera la condivisione su diverse piattaforme di social media (Facebook, Twitter, Instagram, LinkedIn).
   - Autenticazione: includi il test del requisito di essere loggati per condividere (se applicabile).
   - Verifica Contenuto: considera la correttezza del link condiviso e dell'anteprima.
   - Gestione Errori: includi la gestione di potenziali errori durante il processo di condivisione.

Esplora attentamente ciascun ramo di analisi prima di procedere alle conclusioni finali.

5 REFERENCES

Riferimenti bibliografici

[1] M. Abdullah, "An Investigation into Using LLMs for Black Box Testing of Functional Mock-Up Units," 2023. [Online]. Available: https://www.doria.fi/bitstream/handle/10024/192356/mughees_abdullah.pdf?sequence=3&isAllowed=y

[2] A. Author, "Automated test case generation from software requirements using ChatGPT," 2023. [Online]. Available: https://hv.diva-portal.org/smash/get/diva2:1917788/FULLTEXT01.pdf

[3] B. Author, "LLMs for Intelligent Software Testing: A Comparative Study," ACM, 2023. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3659677.3659749?casa_token=ENvQF7EI6wMAAAAA:M4lKueogataW2nGGeZNSYKOvUYch7YgcBroh1Eye9RnguFZYiMZbh-9NneAfoYcbvLje9YNLR4A

[4] M. Chen, et al., "Unleashing the Power of LLMs for Software Engineering," IEEE Transactions on Software Engineering, 2023. [Online]. Available: https://arxiv.org/abs/2301.00021

[5] X. Liu, et al., "A Survey on Large Language Models for Software Engineering," ACM Computing Surveys, 2023. [Online]. Available: https://arxiv.org/abs/2302.00034